Storm Chaser — Speer Technologies Mobile Assessment

Melanie Baratto

September 19, 2026

CONTENTS

Storm Chaser — Speer Technologies Mobile Assessment

- Submission details
- What was built
- Required features
- Bonus features
- Architecture and decisions
- Testing
- Known limits
- AI tool disclosure
- Time log

Storm Chaser — Speer Technologies Mobile Assessment

Submission details

Candidate	Melanie Baratto
Assessment	Mobile — React Native (Senior React Native Developer)
GitHub repository	https://github.com/melaniesigrid/storm-chaser
Screen recording	*[REPLACE WITH GOOGLE DRIVE LINK — set to "anyone with the link"]*

The repository is private, as required. `developer@speer.io` has been invited as a collaborator.

What was built

A React Native application for storm-chasing hobbyist meteorologists, built with Expo SDK 57, Expo Router and TypeScript in strict mode. It fetches live weather for the device's position, scores how interesting the atmosphere actually is, and turns a photograph into a timestamped, geotagged, weather-stamped field report held on the device.

All four required features are complete, along with every one of the seven listed bonus features.

Required features

Requirement	Status	Implementation
Current weather from device location	Complete	Open-Meteo, keyless, live. `expo-location` for the fix.
Meteorological detail for chasers	Complete	Temperature, apparent temperature, wind speed, gusts, bearing, pressure, humidity, cloud cover, precipitation, CAPE and lifted index.
”Not found” view	Complete	A dedicated component with a retry action and a bundled sample-data fallback, reached whenever the fetch cannot resolve.
Camera capture	Complete	`expo-camera`, two-phase flow: photo first, metadata second.
Photo metadata	Complete	Weather conditions, coordinates, accuracy, date and time, notes and classification. Everything but notes and classification is captured automatically and shown read-only for verification.
Local persistence	Complete	Versioned AsyncStorage envelope behind a repository, plus photo files copied into the app document directory.
Navigation between sections	Complete	Expo Router with typed routes: three tabs, a modal capture flow, and detail routes for reports and storm legends.

Bonus features

Senior roles are asked to implement a few of these. All seven are present.

Bonus feature	Implementation
Weather forecast integration	A 12-hour strip and a 7-day outlook, both drawn as charts over shared scaling geometry.
Map visualisation	`react-native-maps`, severity-coloured pins, viewport auto-fitted to your own reports. Three distinct pin shapes for your reports, community reports and storm legends.
Offline functionality	A persisted query cache with a 24-hour horizon, offline and stale banners, and a bundled sample dataset. The field log itself needs no network at all.
Dark mode support	Four themes including two dark ones, system-aware until the user chooses explicitly.
Skeleton screens	Layout-matched skeletons on both data screens.
Pull to refresh	On Conditions and on the field log.
Cloud integration	A signed-upload client and a community feed, resolving through the same `Result` boundary as a real fetch. No keys and no credentials, as the brief requires.

Architecture and decisions

Feature slices over layer folders. `src/features/<domain>/` each own their `model/`, `data/`, `hooks/` and `components/`. `model/` is the bottom layer and contains no React and no I/O, which is why the interesting logic is testable without a renderer or a device.

`Result<T, AppError>` at every boundary. Data access and network code return a result rather than throwing, so callers are forced to handle the failure branch and the UI always has something concrete to render. Error *kinds* — `network`, `permission`, `not_found`, `storage` — map onto specific UI responses rather than a single generic error state.

A repository behind an injected store port. `StormRepository` takes a `KeyValueStore` interface rather than calling AsyncStorage directly, so the whole data layer is testable with an in-memory store and no native modules. Its clock and id generator are injected

for the same reason. Reports live in a single versioned envelope with a forward migration hook.

The weather snapshot is denormalised onto each report. A field log entry has to stay readable years later without re-querying a forecast API that no longer holds historical data for that timestamp, so the conditions at capture are copied onto the record rather than referenced.

Photo first, record second. If the image cannot be copied into permanent storage the mutation stops before a report is written, so the log can never point at a file that is not there. Deletion runs the other way round: the record goes first, because an orphaned file is recoverable and a record pointing at nothing is not.

A theme is data. Colour, typography, motion, brand and atmosphere are all tokens on a theme object. Components never test a theme by name; a theme opts into a behaviour by defining the token for it.

Testing

509 unit tests across 24 suites, run under TypeScript strict mode. They target logic rather than pixels: formatting, the severity model, API mapping, the repository and its migrations, the theme and typography contracts, WCAG contrast, icon mapping, the community layer, storm legends and EXIF date resolution.

The suite runs in two timezones — the default UTC and, via `npm run test:tz`, `America/Toronto` — because a whole class of date bug in this app is invisible under UTC.

```
npm test          # 509 tests
npm run test:tz   # the same suite at a negative UTC offset
npm run typecheck # tsc --noEmit, clean
```

Known limits

Stated plainly, as the brief asks.

- **Cloud sync is the client half only.** There is no backend and nothing is uploaded. `syncState` is modelled and persisted but only ever moves when a configured sync target exists. This is what the brief specifies, but it means the feature is unexercised.
- **The network probe polls every 15 seconds** rather than subscribing to an event stream. A production build would use NetInfo with TanStack Query's `onlineManager`.
- **No component rendering tests.** The suite has no renderer, which is why things like contrast and icon mapping are asserted as pure functions instead. This is the clearest gap in the test strategy.
- **The community feed is fictional.** 39 bundled sample reports, badged as demo data everywhere they appear, because there is no server to fetch real ones from.
- **Legend photography is illustrative in most cases.** Freely licensed photographs of specific historical storms largely do not exist, so many images show the phenomenon rather than the event. Each one prints a caption saying what it actually shows.

AI tool disclosure

The assessment permits AI assistance and requires disclosure. The full account is in `docs/AI_DISCLOSURE.md`, which names the tools, separates what was directed from what was generated, and lists specific cases where the model's output was wrong and was corrected.

In summary: Claude and Claude Code were used throughout as an implementation partner. The architecture, the product decisions, the data model and the acceptance bar were mine; a large share of the implementation was generated to that direction and then reviewed, run and corrected. No other AI tools were used, and no code was copied from an existing weather or storm-chasing application.

Time log

Total: **10 h 15 m**, inside the 24-hour window. The full session-by-session breakdown is in `docs/TIME_LOG.md`.

Session	Duration	Work
One — build	7 h 45 m	Stack, design system, weather domain and feature, storm capture, persistence, field log, map, offline, dark mode, cloud client, tests, documentation.
Two — themes and accessibility	1 h 20 m	The theme system, brand identity, onboarding, landing page, and three accessibility defects closed.
Three — community, legends and charts	1 h 10 m	Community layer, 22 storm legends, the chart primitives, brand marks, and a pre-landing review pass.

Where git history can bound a session, it does, and the log says so. Session one landed as a single commit and its figures are estimates.